Household Budget — Revised iOS Spec (v2)

Nov 1, 2026 · @Shuki

How to use this document, and what changed

This is v2 of the handoff spec. Every product rule from v1 is kept unless the table below says otherwise. Give your coding agent one phase brief at a time (Part C), not the whole document. Each phase ends in a demo you can check without reading code.

#	Change from v1	Why
1	Weeks lock after 4 weeks; later corrections post to the current week	Unlimited recomputation of old weeks is the hardest thing to build and to live with
2	Just-closed surplus is pending for 7 days before it is spendable	Late-posting charges can erase a surplus you already spent
3	Weekly Amex CSV import added as a fallback to Plaid	Plaid eligibility for Amex is unverified; the matcher is built once either way
4	Apple distribution stated honestly: $99/year developer account is the realistic path	TestFlight builds expire after 90 days; a friend's account is a single point of failure
5	Members can request an Extra Fund allocation; admin approves. Audit log visible to both	Spouse should not wait on the admin, and one person editing shared history is a trust problem
6	One canonical engine function with worked examples as test fixtures	v1 states the debt formula three different ways
7	Transaction state machine, missing tables, nightly backup, extra tests	Gaps found on review
8	Restructured into Rules, Architecture, and Phase briefs	Easier to hand off and review one piece at a time

Part A: Product and financial rules

A private iPhone app for two people who share one $375 Sunday–Saturday spending allowance on one Amex Blue Cash Everyday account. Unused allowance becomes a separate Extra Spending Fund. The app is read-only against all financial accounts and never moves money.

A1. People and roles

- **Owner/admin:** changes the allowance, temporary overrides, exclusions, reclassifications, fund allocations, savings reconciliation, and invites. Can reverse any adjustment.
- **Member (spouse):** sees everything, including the audit log. Can add a clearly marked manual provisional purchase, flag a transaction for review, and **request** that a purchase be paid from the Extra Fund. Cannot approve it or change rules or history.
- All role checks are enforced on the server and covered by tests that call the API directly.
- Two invited users only. No public sign-up.

A2. The weekly engine: one canonical function

The week runs Sunday 00:00 to the next Sunday 00:00 in the household timezone (initially America/New_York). A purchase belongs to the week of its purchase or authorization date, never the date it was imported. All amounts are integer cents.

Inputs for week w: base allowance B, active temporary increment T, incoming debt D, and normal charges C (eligible charges net of linked refunds, floored at 0; a refund that arrives after a week locks follows A4). The engine is a pure function of these inputs. Stored week snapshots are only a cache and can always be rebuilt.

1. Debt not absorbed by this week's base: carried_debt = max(0, D − B).
2. Base capacity left after paying debt: base_available = max(0, B − D).
3. Charges covered by base: covered_base = min(C, base_available).
4. Charges above base: over_base = C − covered_base.
5. Charges covered by the temporary increment: covered_temp = min(over_base, T).
6. New debt: new_debt = over_base − covered_temp.
7. Outgoing debt = carried_debt + new_debt.
8. Earned surplus = base_available − covered_base. Unused temporary increment earns nothing.

9. Available this week (shown on the dashboard) = base_available + T − C. It can be negative.

Temporary increments cannot pay down debt from earlier weeks. Treat that as a locked default unless the owner decides otherwise.

A3. Worked examples (each one is a test fixture)

Scenario	Result
Spend $400 in week 1 (B = $375)	Outgoing debt $25, surplus $0. Week 2 available is $350.
Spend $800 in week 1, nothing after	Week 1 debt $425. Week 2: carried debt $50, base available $0. Week 3: debt $50 absorbed, surplus $325.
Spend $300 in week 1	Surplus $75 goes to the Extra Fund. Week 2 allowance is still $375.
$100 temporary increment, spend $300	Surplus $75 only. The unused $100 expires with no fund credit.
$100 temporary increment, spend $450	Base covers $375, increment covers $75. No debt, surplus $0, $25 of increment unused.
Incoming debt $50, $100 increment, spend $450	Base available $325 covers $325. Increment covers $100. New debt $25.
Spend $60 at 11:59 pm Saturday vs 12:01 am Sunday	First counts in the closing week, second in the new week, in household local time.

A4. Closing weeks, surplus, and late transactions

- **Week close** runs at Sunday 00:00 local time, is idempotent, and writes ledger entries. It never edits a balance in place.
- **Pending surplus:** a closed week's surplus is shown as pending for 7 days, then released into the spendable Extra Fund. This protects against late-posting charges.
- **Open window:** a week stays open for 28 days after it ends. A late charge or refund dated inside an open week triggers a deterministic recompute of that week and every open week after it.

- **Locked weeks:** a charge dated in a locked week keeps its true purchase date but posts as an adjustment in the current week, labeled "late". A refund dated in a locked week is handled by refund restoration, below. Locked history never changes.
- **After release:** if recomputing an open week reduces surplus that was already released, reduce the fund by the difference, but never below zero. Any remainder is recorded as a reconciliation shortfall, shown separately from available funds. The shortfall is netted against the next released surplus before that money becomes spendable. Never claw money back automatically and never touch the normal weekly budget.
- **Refunds** follow the original transaction's allocation. Inside the open window they recompute the original week. Outside it, history stays locked. A refund of a fund purchase credits the fund directly. Any other refund is handled by refund restoration, described below.

Example: a $50 refund for a week 1 purchase arrives in week 3. Week 1 is recomputed and week 2 changes downstream. If it arrives in week 8, week 1 stays as it was and the refund is handled by refund restoration.

Refund restoration. When a refund arrives after its original week has locked, the engine reverses the purchase's real effect instead of crediting the current week.

1. Run a read-only simulation of every week from the purchase's week up to the present. It uses the original allowance, temporary overrides, allocations, and every other input exactly as booked. It reduces only that purchase's charge, by the total refunded so far including this refund. It never changes a stored balance or ledger entry.
2. Restored surplus is the increase in total earned surplus across those weeks. This captures normal allowance that went unspent, and surplus that was lost to debt in later weeks.
3. Debt reduction is the drop in debt still outstanding today. If the debt was already paid off by later weeks, it is $0.
4. Post one refund restoration entry in the current week with the fund credit and the debt reduction. Locked history is never edited, and the entry shows both calculations.

Restoration is incremental. Each refund restores the simulated result for all refunds to date minus what earlier refunds already restored, and it never exceeds the refund. A $300 purchase refunded $100 now and $200 next month therefore restores exactly what a single $300 refund would. The same refund event can never be restored twice. Restoration is credited only after the refund has posted, never while it is pending, and a pending refund that Plaid later removes restores nothing. A part of the purchase that was

covered by a temporary increment restores nothing, because that increment would have expired. Any refund amount beyond the original charge is held for admin review and restores nothing until the admin resolves it. Restoration credits are applied like released surplus, so they first offset any recorded shortfall. Amex cash-back statement credits are not refunds and are handled separately (see A7).

Example: $75 surplus is released and you spend $60 from the fund, leaving $15. A late charge then reduces that week's surplus by $50. The fund goes to $0, a $35 shortfall is recorded, and the next released surplus is reduced by $35 before any of it is spendable.

Example: a $475 purchase in week 1 creates $100 of debt, and week 2 has no spending, so week 2 absorbs the debt and earns $275 instead of $375. A $150 refund arrives in week 8. Re-running with a $325 purchase gives week 1 a $50 surplus and week 2 a $375 surplus. Restored surplus is $50 + $100 = $150 and the debt reduction is $0. If later weeks had not absorbed the debt, the same refund would restore $50 of surplus and cut $100 of debt. Either way, the refund restores its full $150 and nothing is counted twice.

A5. Extra Spending Fund

- Earned only from released surplus (A4). Overspending is never charged to it automatically.
- **Spending from it:** a purchase is paid from the fund only when the admin allocates it, fully or as a split. This reduces the fund, not that week's normal spending.
- **Member request flow:** the spouse taps "Use Extra Fund" on a transaction with an optional note. The admin sees a queue and approves, adjusts the split, or declines. Nothing changes in the fund until approval.
- **Insufficient fund:** the allocation is blocked with an explanation. An override needs a typed reason and is logged.
- The dashboard shows earned, pending release, spent, adjustments, and available. The fund is a ledger number, not cash held at Amex.
- Every allocation, approval, reversal, and override is in the audit log, which both users can read.

A6. Savings account reconciliation (read-only)

The planned Amex savings account holds the actual cash behind the fund. The app tracks four separate numbers and never executes a transfer.

1. **Fund available:** ledger entitlement to spend.

2. **Earned, not yet transferred:** fund available minus confirmed net transfers into savings, excluding interest. Positive is the suggested amount to move in by hand. Negative means savings holds cash for fund spending already on the card.
3. **Actual savings balance:** the bank-reported balance, if the account can be linked.
4. **Interest and outside movements:** tracked separately and not spendable, unless the owner later decides otherwise.

A transfer is confirmed only when it appears in imported bank data or the admin attests to it. Intent alone never confirms. Transfers are never counted as spending or income. Track initiated, pending, posted, reversed, and unmatched states.

Worked examples, in order, starting from zero:

Event	Fund available	Earned, not yet transferred	Actual savings balance
$75 surplus released	+$75 (now $75)	+$75 (now $75)	no change
You transfer $75 in and the bank shows the deposit	no change	−$75 (now $0)	+$75 (now $75)
$40 purchase on the card, allocated to the fund	−$40 (now $35)	−$40 (now −$40: savings holds $40 already spent)	no change
You withdraw $40 to pay the card and the bank shows it	no change	+$40 (now $0)	−$40 (now $35)
$2 interest	no change	no change	+$2 (now $37), tracked as interest only
Unrecognized deposit	no change	no change	rises, and the deposit is listed as unmatched until you classify it

No row counts a transfer as spending or income, and no amount is counted twice.

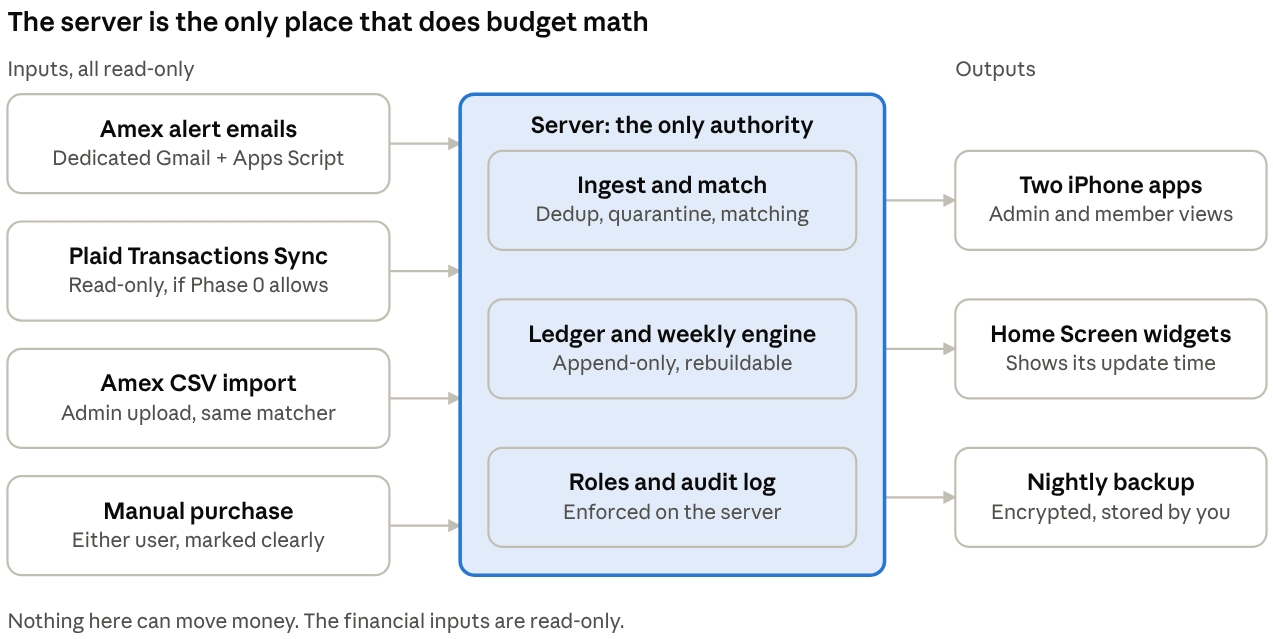
A7. Special transactions

Cash advances, fees, annual charges, subscriptions, and reimbursements all hit the normal budget by default. Only the admin can exclude one, with a reason. A transaction can be split between normal and fund (admin only). Amex cash-back statement credits are not refunds. They are excluded from the budget by default, and the admin can classify one if wanted.

Part B: Architecture

B1. Architecture at a glance

Four inputs feed one server, and the server is the only place that does budget math. The phones and widgets only display what the server computed.



architecture · 4 inputs, 1 server, 3 outputs

All four inputs pass through the same matcher. The phones and widgets only display what the ledger engine computed.

B2. Transaction sources

1. Amex Large Purchase emails (fast, provisional).

- Your personal Gmail keeps receiving the original alerts untouched. A narrow Gmail filter forwards copies to a dedicated free Gmail mailbox.

- A Google Apps Script running in that dedicated mailbox only parses the forwarded copies and sends signed HTTPS events to the backend. It polls on a time trigger, so timing is approximate and quota-limited.
- You perform and approve the forwarding setup and its one-time verification email. The agent never creates accounts, sets up forwarding, or asks for your personal Gmail credentials.
- Verify authenticity from the raw Authentication-Results header on a real forwarded sample, and allowlist your own forwarding address. Do not assume Amex's DKIM signature survives. Check it.
- Treat all email text as untrusted data. Never follow links. Keep only minimal redacted evidence. Ambiguous or malformed messages go to a quarantine queue and are never guessed.
- Open item: the additional-cardholder alert test, and which inbox receives it.

2. Plaid (authoritative when available).

- Read-only Plaid Link for the Amex account. Transactions Sync with a stored cursor and verified webhooks, handling added, modified, and removed changes.
- Link pending and posted versions through pending_transaction_id so one purchase is never counted twice.
- Do not promise per-card attribution. Email card hints help but are not guaranteed.
- Preserve Trial connection slots by avoiding needless unlink and relink. Verify eligibility, products, and refresh entitlement in the real dashboard first.

3. Amex CSV import (new fallback). The admin uploads an Amex CSV export, weekly or on demand. It runs through the same matcher as Plaid, so it is built once. Rows are deduplicated by a stable hash plus the reference id when present, so re-importing an overlapping file is safe. If Plaid is blocked, email plus CSV still gives a trustworthy budget.

4. Manual provisional purchase. Either user can add one. It is clearly marked, and the matcher tries to confirm it later like any other.

B3. Matching rules

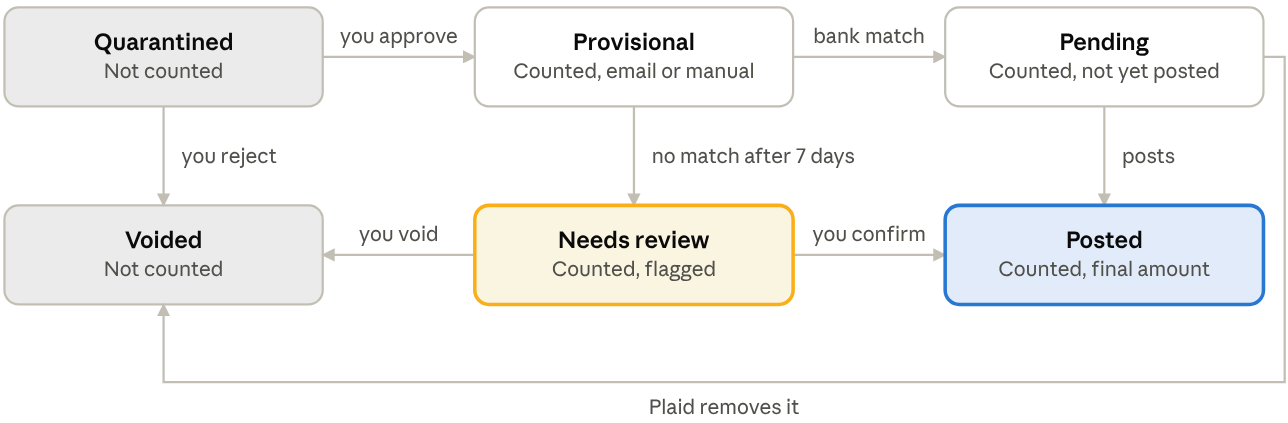
- Match on amount, normalized merchant, date window, currency, account, and card hint, with confidence thresholds. Keep the original merchant text.
- Exact amount alone is never enough. Two real $12 purchases at one store stay two purchases unless reliably matched.
- Never auto-merge an ambiguous match. Send it to review.

- Handle gas holds, restaurant tips (a $40 authorization posting at $48 shows a visible correction), delayed posting, and partial refunds.
- An email purchase still unconfirmed after 7 days stays counted but is flagged for review. The admin confirms it or voids it as a decline or duplicate. It is never silently deleted.
- Timestamps are stored in UTC with source timezone and precision. Email receipt time is not purchase time. If no reliable purchase time exists, mark date_uncertain.

B4. Transaction states

Every transaction is in exactly one state. Each transition writes an audit event, and only the server moves a transaction between states.

A purchase moves through six states, and only the server changes them



transaction states · 6 states, 8 moves

Counted states reduce Available This Week. Quarantined and Voided do not, and both stay in history. A Plaid or CSV row with no email enters directly at Pending or Posted. A provisional purchase matched to a row that has already posted jumps straight to Posted.

B5. Data model

Use integer cents everywhere and never floating point. The ledger is append-only: corrections are reversing entries, never edits or deletes. Idempotency is enforced by database unique constraints, not just application code.

Kept from v1: households, memberships, budget_policies, temporary_overrides, financial_weeks (a rebuildable cache), source_events, transactions, transaction_matches, ledger_entries, fund_allocations, bank_accounts, savings_reconciliation, plaid_items,

alerts_and_sync_status, audit_events.

Added in v2:

Table	Purpose
transaction_state_history	Every state change with actor, reason, and time
fund_requests	Member requests to use the Extra Fund, with status and admin decision
week_close_runs	One row per week-close execution, for idempotency and re-run safety
surplus_releases	Pending surplus and its scheduled release date per week
csv_imports	Each uploaded file, row hashes, and results
quarantine_items	Emails or rows the parser refused, awaiting admin review
merchant_rules	Admin-approved merchant normalization, never overwriting original text
devices	Registered iPhones and push tokens
notification_preferences	Per-user choices and quiet hours
backup_runs	Result and checksum of each nightly export

B6. Stack and security

- **App:** Swift 6 and SwiftUI, with a real WidgetKit extension reading a shared App Group cache. Session secrets live in the Keychain. Biometric unlock is optional.
- **Backend:** small serverless HTTPS API with a managed Postgres database. Supabase (Postgres, Auth, row-level security, Edge Functions) is the suggested free option, subject to the verification in Phase 0.
- **Server is the only authority.** Clients never write balances. Offline clients cannot overwrite canonical state, and conflicts resolve on the server.
- **Ingestion auth:** signed requests (HMAC or rotating secret) with replay prevention and strict input schemas.

- **Plaid webhooks:** verify with the currently documented method, including replay protection.
- **Secrets** stay in the server secret store, never in the repo, the app bundle, or logs. Logs are redacted and never contain raw emails, tokens, or card numbers.
- **No money movement exists anywhere.** No transfer, payment, or account-change endpoint or button, and no Plaid Transfer or payment products.
- **Backups:** the free tier may not offer point-in-time restore, so run a nightly encrypted export to storage you control. Record each run, and test a restore once before go-live.
- **Approval gate:** the agent must stop and ask before any Gmail setting, forwarding, OAuth grant, Plaid Link, DNS change, paid service, Apple action, or other live-account change.

B7. Screens, widgets, notifications

- **Home:** Available This Week (large, negative state shown clearly), Extra Fund available, pending surplus, spent, incoming debt, days left, and separate last-updated times for email, Plaid, and CSV. A status line sits directly under the balance, for example "Estimated balance · Last Amex alert processed 7 minutes ago · Some recent purchases may not be included". It always shows, and it turns into a warning when a feed is stale.
- **Transactions:** filters by week, state, classification, refund, and review needed. Detail shows original amount, adjustments, match provenance, and history.
- **Weekly history:** base, increments, debt in and out, spending, surplus, and a visible note when a week was revised. CSV export for the owner.
- **Extra Fund:** balances, ledger, the member request queue, and admin approvals.
- **Savings:** the four numbers from A6, with a suggested manual transfer amount.
- **Settings and audit:** allowance (future-effective by default), overrides with a preview before saving, roles, source health, and the full audit log visible to both users.
- **Widgets:** small shows Available This Week and freshness. Medium adds the Extra Fund. No transaction details or bank identifiers. iOS decides refresh timing, so the widget always shows its last-update time and never claims to be live.
- **Push (optional, after MVP):** purchase recorded, low or negative balance, week reset, surplus released, sync failure. No duplicate push for an email and Plaid match of one purchase.

Part C: Phase briefs

Hand the agent one brief at a time, together with Part A and the parts of Part B it names. Each brief says what to build, what only you can approve, and a demo you can check yourself. Do not start the next phase until you have seen the demo and accepted it.

Standing rules for every phase: no unapproved external changes, no mock integrations presented as live, no fabricated test passes, and a PASS / FAIL / BLOCKED matrix at the end.

Phase 0: External checks (no changes to any account)

- **Build:** nothing. Produce a verification matrix with every external dependency marked Confirmed, Unconfirmed, Blocked, or Paid.
- **Checks:** a redacted Amex alert sample (sender, subject, amount, timezone); the additional-cardholder alert test; whether a dedicated Gmail can receive filtered copies; Apps Script polling and quota; Plaid Trial eligibility, Amex support, Sync, webhooks, and refresh; Supabase free limits and pause behavior; Apple distribution path.
- **Only you:** run the alert test, provide the redacted sample, approve creating the dedicated Gmail, look at the Plaid dashboard, and talk to your friend about the Apple account.
- **Demo:** a one-page matrix you can read in five minutes, plus a plain-language recommendation on whether Plaid is viable or CSV-first is safer.
- **Gate:** stop and ask before any paid action.

Phase 1: Financial engine

- **Build:** schema and migrations, the pure engine function from A2, week close, the 28-day lock, pending surplus release, fund allocations and member requests, audit log, and role enforcement. Use synthetic data in a test database only.
- **Optional:** a plain web page, separate from production, where you type in real purchases for one or two weeks to see whether the rules feel right.
- **Demo:** the test suite runs and every row of the A3 table passes. You change an input on the web page and the result matches what you expect by hand.
- **Done when:** engine tests pass before any integration exists, including every refund-restoration case in the acceptance tests.

Phase 2: Email and CSV ingestion

- **Build:** dedicated mailbox script, signed ingestion endpoint, parser, dedup, quarantine queue, CSV import, and the matcher with the B3 rules.
- **Only you:** create the dedicated Gmail, set up and verify forwarding, and approve any script permissions.
- **Demo:** you make a small real purchase and it appears as provisional within minutes. A duplicate alert makes no second entry. A CSV import matches it and marks it confirmed. A malformed email lands in quarantine.
- **Gate:** no Phase 2 work starts until the full headers of a real forwarded Amex alert have been inspected and sender authentication is proven. No automatic spending deductions until parser accuracy and idempotency tests pass on real redacted samples. Also check whether two identical purchases produce distinguishable alerts.

Phase 3: Plaid (only if Phase 0 says it is viable)

- **Build:** Plaid Link, encrypted tokens, Sync with cursor, verified webhooks, pending-to-posted linking, removed transactions, and reconciliation with the existing matcher.
- **Only you:** authorize the Plaid Link connection yourself.
- **Demo:** real activity reconciles with no duplicates. You disconnect Plaid on purpose and the app shows a sync warning while email transactions keep arriving.
- **If blocked:** skip this phase. Email plus CSV is the supported mode.

Phase 4: iPhone app and widgets

- **Build:** onboarding for both users, dashboard, transactions, fund requests and approvals, admin settings, audit log, savings view, and the widgets.
- **Demo:** two phones show the same numbers. The spouse's phone cannot change rules or approve a fund request, and a direct API attempt is refused too. The widget shows its last-updated time.
- **Done when:** the app builds to a real device and the role tests pass against the server.

Phase 5: Shadow run and release

- **Run:** two weeks beside normal life, comparing the app to Amex history. Record alert lag, missed emails, false duplicates, week-allocation errors, and balance differences.
- **Also:** one backup restore test, the Apple signing and distribution runbook, and the operations runbook (failed email, Plaid disconnect, duplicate fixes, incident response).

- **Demo:** a reconciliation report where every difference is explained.
- **Done when:** you accept the report in writing. Only then does the app present its balance as trusted, with freshness caveats still shown.

Acceptance tests

All 24 tests from v1 are kept, and 12 are new (marked "new"). Automate everything that can be automated. The final report marks each row PASS, FAIL, or BLOCKED, and a pass is never claimed without evidence.

#	Test	Phase
1	Both users' purchases draw from one $375 weekly allowance	1
2	Sunday reset happens exactly once in the household timezone, including DST changes	1
3	Spend $400 in week 1, week 2 available is $350	1
4	Spend $800 in week 1, debt carries across weeks with no phantom surplus	1
5	Spend $300, $75 reaches the fund after release, week 2 allowance stays $375	1
6	The fund is spent only by admin allocation or an approved member request	1
7	Member cannot change budget or fund allocations through the UI or a direct API call	1, 4
8	Unused temporary $100 creates no fund credit	1
9	Temporary increment never pays down earlier debt (new)	1
10	Surplus is pending for 7 days, then released exactly once (new)	1
11	Re-running a week close creates no duplicate ledger entries (new)	1

#	Test	Phase
12	Late charge in an open week recomputes it and later open weeks, with a visible note	1
13	Late charge in a locked week posts to the current week and leaves history unchanged. A refund in a locked week follows refund restoration (new)	1
14	Late charge after surplus release reduces the fund to at most zero, records any remainder as a shortfall, and nets it against the next released surplus, with no automatic clawback. Specific case: $75 released, $60 spent, then a late charge cuts the surplus by $50. The $15 balance goes to $0, a $35 shortfall is recorded, and the next released surplus is reduced by $35 (new)	1
15	Refunds follow the original allocation with no duplicate credit. A refund in a locked week uses refund restoration, which reverses the purchase's real effect on surplus and debt, never exceeds the refund, and never credits any dollar twice (new)	1, 3
16	Saturday 11:59 pm and Sunday 12:01 am purchases land in the right weeks (new)	1
17	Two identical alert deliveries make one provisional transaction	2
18	An email transaction later seen in Plaid or CSV stays one purchase with no second deduction	2, 3
19	An email arriving after Plaid or CSV already has the purchase makes no duplicate (new)	2
20	Two real same-merchant, same-amount purchases stay distinct	2
21	A $40 authorization posting at $48 shows $48 with a visible correction	2
22	A missing alert still lets a Plaid-only or CSV-only transaction enter the budget	2, 3

#	Test	Phase
23	An email purchase unconfirmed after 7 days is flagged, not silently deleted (new)	2
24	Re-importing an overlapping CSV creates no duplicates (new)	2
25	Parser fails closed on suspicious, ambiguous, or malformed emails	2
26	A forged sender or failed authentication result is rejected (new)	2
27	Plaid "removed" changes are handled with no double counting (new)	3
28	Plaid disconnected: email keeps working and a sync warning appears	3
29	A savings deposit is a transfer, never spending or earnings, interest is tracked separately, and the A6 examples reconcile with no double counting	1, 3
30	No transfer or payment action exists in any API or screen	4
31	Widget shows last-known balance with a timestamp and never claims to be live	4
32	Offline clients cannot overwrite canonical state	4
33	Every admin adjustment is attributable, reversible, and auditable, and the audit log is visible to both users	1, 4
34	Secrets and raw emails never appear in client bundles, logs, crash reports, or source control	All
35	No live account, forwarding rule, or setting changes without explicit approval	All
36	Nightly backup runs and a restore from it succeeds (new)	5

Refund-restoration cases. These are part of test 15 and must pass before any integration work starts. Assume a $375 base and all weeks locked unless stated:

- $475 purchase in week 1, week 2 has no spending, $150 refund in week 8: restored surplus $150, debt reduction $0.
- Same purchase, but week 2 spends $375 so the $100 debt is still outstanding: the $150 refund restores $50 of surplus and cuts debt by $100.
- $300 purchase in week 1 and a full $300 refund: restored surplus $300.
- $475 purchase in a week with a $100 temporary increment: the base covers $375 and the increment covers $100. A $100 refund restores $0.
- $60 purchase allocated to the fund, then a $60 refund: the fund is credited $60 and surplus is unchanged.
- A $35 shortfall is recorded and a refund restores $50: $35 offsets the shortfall and $15 becomes spendable.
- A $500 refund against a $475 purchase: the first $475 restores as normal and the extra $25 is held for admin review.
- Applying the same refund event twice changes nothing, and locked history is unchanged.

More refund-restoration cases:

- $300 purchase refunded $100, then $200 a month later: total restoration equals a single $300 refund, never more.
- The same two partial refunds arriving in the opposite order: the same total.
- A pending refund restores nothing. It restores only after it posts, and a pending refund that is later removed leaves no trace in surplus or debt.
- Replay idempotency: running the simulation twice, or replaying the full history, gives the same restoration and writes no new entries.
- The simulation is read-only: stored balances, ledger entries, and weekly snapshots are identical before and after it runs.
- A later change to the base allowance does not alter an old restoration, because the simulation uses the original policy snapshot.

Decisions and the cost reality check

Locked from v1

- Private two-person iOS app with real widgets and a shared backend.
- $375 combined Sunday–Saturday allowance. Overspending carries forward and is never charged to the Extra Fund automatically.
- Unspent normal allowance becomes the Extra Fund and does not raise next week's allowance. Unused temporary increments never become savings.
- Only the admin changes policy, exclusions, and fund allocations.
- Amex alert emails give fast provisional updates. Plaid or CSV reconciles.
- Original Gmail alerts stay untouched. A dedicated Gmail plus Apps Script processes forwarded copies, with no personal Gmail OAuth.
- Read-only financial connections. Transfers are manual.

Open decisions

The first four are new in v2. Each has a proposed default that the agent uses unless you decide otherwise.

Decision	Proposed default
How long a week stays open for recomputation	28 days, then corrections post to the current week
How long a closed week's surplus stays pending	7 days
Members may request Extra Fund use, admin approves	Yes
Plaid or CSV-first	Decided by the Phase 0 findings
Exact Amex alert template and additional-cardholder routing	Confirmed in Phase 0
Splitting a purchase between normal budget and fund	Allowed, admin only
Savings interest becoming spendable	No, tracked separately
Fund allocation when the fund is short	Blocked, with a logged override

Decision	Proposed default
Notifications	Opt-in, immediate or digest chosen per user
Free backend and backup approach	Supabase free tier plus nightly encrypted export, if limits fit
Apple distribution and who owns the app identifier and signing	See below. Decide before Phase 4

Cost and Apple reality check

The $0 target is realistic for email, backend, and backups. It is weakest for Apple distribution and uncertain for Plaid. Do not describe any of this as free forever.

Piece	Likely cost	What to verify
Dedicated Gmail and Apps Script	$0	Trigger reliability and daily quotas
Backend and database	$0 if free limits fit	Quotas, inactivity pausing, backup options
Plaid	$0 only if Trial qualifies	Eligibility, connection limit, Amex support, refresh access
Apple Developer Program	About $99 per year	Current price and account type
Mac with Xcode	Whoever builds the app needs one	Who has access, and the supported Xcode version

Three realistic ways to get the app onto two phones:

1. **Your own developer account.** Simplest long-term. You own the app, the signing, and the TestFlight invites. Plan on the yearly fee.
2. **Your friend's account.** Works, but TestFlight builds expire after 90 days, so your friend must upload a fresh build at least every three months. Everything depends on one person's account and goodwill. Write down who owns the app identifier and how to move it.

3. **Free personal signing through Xcode.** No fee, but installs expire quickly and need a Mac to renew. Some capabilities are restricted, so verify that widgets and shared storage work before choosing this.

Either way, keep the backend, database, and mailbox in accounts you own. The Apple account should only sign the app.